

Лабораторная работа №4. Flutter: списки, модели данных и карточки

Цель работы: научиться хранить данные в виде модели, строить прокручиваемые списки через **ListView.builder** и создавать переиспользуемые карточки — на примере каталога аниме.

Приложение, которое мы с вами создадим, похоже на то, что создали в Android Studio (прокручиваемый список):



Часть 1. Подготовка проекта

Шаг 1. Создание проекта

1. Перейдите в папку с вашими проектами

Выполните в терминале:

```
flutter create anime_catalog
```

```
cd anime_catalog
```

```
mkdir img
```

```
code .
```

2. Запустите приложение и убедитесь, что всё работает:

flutter run -d chrome

3. Нажмите **q**, чтобы остановить приложение.

Шаг 2. Подключение assets

1. Скачайте архив с постерами аниме с сайта, **создайте** и поместите картинки в папку **assets/images/**.

Структура папки:

assets/images/

- |— anime1.jpg ← Death Note
- |— anime2.jpg ← Attack on Titan
- |— anime3.jpg ← Naruto
- |— anime4.jpg ← One Piece
- |— anime5.jpg ← Demon Slayer
- |— anime6.jpg ← My Hero Academia
- |— anime7.jpg ← Sword Art Online
- |— anime8.jpg ← Fullmetal Alchemist: Brotherhood
- |— anime9.jpg ← Tokyo Ghoul
- |— anime10.jpg ← Steins;Gate

2. Удалите содержимое файла **lib/main.dart** и файл **test/widget_test.dart**

3. Откройте **pubspec.yaml**, удалите ненужные комментарии и зарегистрируйте папку целиком — чтобы не перечислять каждый файл отдельно:

```
flutter:
  uses-material-design: true
  assets:
    [+ - assets/images/
```

Если указать папку со слешем в конце — Flutter подключит все файлы внутри автоматически — это удобно, когда файлов много.

4. Сохраните и выполните:

flutter pub get

Шаг 3. Инициализация Git и подключение к GitHub

Выполните в терминале:

git init -b main

Добавьте в конец файла `.gitignore`:

android/

ios/

linux/

macos/

windows/

.idea/

***.iml**

.DS_Store

Thumbs.db

Выполните в терминале:

git add .

git commit -m "Initial commit: create Flutter project with assets"

Создайте публичный репозиторий **Flutter_Lab4** на GitHub и привяжите:

git remote add origin <URL_вашего_репозитория>

git push -u origin main



СКРИНШОТ 1: Скриншот терминала с результатом **git push**. Сохраните в `img/step1_ВашаФамилия.png`

Часть 2. Модель данных

Шаг 1. Что такое модель данных

До сих пор мы хранили данные в простых переменных: **String**, **int**, **List<String>**. Но когда данные сложнее — например, у каждого аниме есть название, описание, жанр, рейтинг и год — удобнее хранить всё это вместе в одном объекте.

Для этого создаётся **класс-модель** — обычный Dart-класс без наследования от виджетов, который просто описывает структуру данных. Это аналог **data class** в **Kotlin** или класса с автосвойствами в **C#**.

Шаг 2. Создаём модель **Anime**

1. Создайте папку `lib/models/` и в ней файл `lib/models/anime.dart`

2. Откройте его и напишите:

```
class Anime {
  const Anime({
    required this.title,
    required this.description,
    required this.genre,
    required this.year,
    required this.rating,
    required this.imagePath,
  });

  final String title;
  final String description;
  final String genre;
  final int year;
  final double rating;
  final String imagePath;
}
```

- Все поля `final` — объект создаётся один раз и не меняется. Это аналог `data class` в Kotlin с `val` полями.
- Конструктор с именованными параметрами (`{}`). Все параметры `required` — нельзя создать `Anime` не передав все поля.
- `const` конструктор — если все значения известны на этапе компиляции, Flutter оптимизирует создание объекта.

Шаг 3. Создаём список данных

Создайте файл `lib/models/anime_data.dart` — здесь будут жить все наши данные.

```
import 'anime.dart';

const animeList = [
  Anime(
    title: 'Death Note',
    description:
      'Гениальный студент находит таинственную тетрадь смерти и решает создать идеальный мир.',
    genre: 'Триллер, Психология',
    year: 2006,
    rating: 9.0,
    imagePath: 'assets/images/anime1.jpg',
  ),
];
```

Текст для вставки:

1. 'Death Note',

Гениальный студент находит таинственную тетрадь смерти и решает создать идеальный мир.',

'Триллер, Психология',

2006,

9.0,

imagePath: 'assets/images/anime1.jpg',

Аналогично вам нужно добавить следующие аниме:

2. 'Attack on Titan',

'Человечество борется за выживание против гигантских титанов, пожирающих людей.',

'Экшен, Драма',

2013,

9.1,

imagePath: 'assets/images/anime2.jpg',

Добавляем после запятой:

```
const animeList = [  
  Anime( ...  
  Anime(  
    title: 'Attack on Titan',  
    description:  
      'Человечество борется за выживание против гигантских  
титанов, пожирающих людей.',  
    genre: 'Экшен, Драма',  
    year: 2013,  
    rating: 9.1,  
    imagePath: 'assets/images/anime2.jpg',  
  ),  
];
```

По аналогии добавьте следующие 8 аниме:

3. 'Naruto',

'История молодого ниндзя Наруто, мечтающего стать Хокаге своей деревни.',

'Экшен, Приключения',

2002,

8.3,

imagePath: 'assets/images/anime3.jpg',

4. 'One Piece',

'Приключения Луффи и его пиратской команды в поисках легендарного сокровища.',

'Приключения, Комедия',

1999,

8.9,

imagePath: 'assets/images/anime4.jpg',

5. 'Demon Slayer',

'Танджиро сражается с демонами, чтобы спасти свою сестру и отомстить за семью.',

'Экшен, Фэнтези',

2019,

8.7,

imagePath: 'assets/images/anime5.jpg',

6. 'My Hero Academia',

'В мире супергероев подросток без способностей мечтает стать величайшим героем.',

'Экшен, Супергерои',

2016,

8.4,

imagePath: 'assets/images/anime6.jpg',

7. 'Sword Art Online',

'Игроки оказываются в ловушке виртуальной реальности, где смерть в игре означает смерть в реальности.',

'Фэнтези, Приключения',

2012,

7.7,

imagePath: 'assets/images/anime7.jpg',

8. 'Fullmetal Alchemist: Brotherhood',

'Два брата-алхимика ищут философский камень, чтобы вернуть утраченные тела.',

'Экшен, Фэнтези',

2009,

9.1,

imagePath: 'assets/images/anime8.jpg',

9. 'Tokyo Ghoul',

'Студент Канеки превращается в полу-гуля после трагического инцидента.',

'Ужасы, Экшен',

2014,

7.9,

imagePath: 'assets/images/anime9.jpg',

10. 'Steins;Gate',

'Учёный случайно изобретает машину времени и сталкивается с последствиями.',

'Sci-Fi, Триллер',

2011,

9.1,

imagePath: 'assets/images/anime10.jpg',



animeList объявлен как **const** — весь список создаётся на этапе компиляции и никогда не меняется. Это эффективнее, чем **final**: объект создаётся один раз на этапе компиляции и не перевыделяется при перерисовке.



СКРИНШОТ 2: Скриншот структуры папки **lib/** с созданной папкой **models/** и двумя файлами внутри. Сохраните в **img/step2_ВашаФамилия.png**

Выполните в терминале:

git add .

git commit -m "Add Anime model and data list"

Часть 3. Виджет карточки

Шаг 1. Создаём **AnimeCard**

Создайте файл **lib/anime_card.dart**. Строим карточку пошагово.

Card — виджет Material Design. Даёт карточке тень и скруглённые углы автоматически. **clipBehavior: Clip.hardEdge** обрезает дочерние виджеты по границам карточки — без этого картинка будет вылезать за скруглённые углы.

Шаг 1.1 — Базовая структура:

```
import 'package:flutter/material.dart';
import 'models/anime.dart';

class AnimeCard extends StatelessWidget {
  const AnimeCard({super.key, required this.anime});
  final Anime anime;

  @override Widget build(BuildContext context) {
    return Card(
      clipBehavior: Clip.hardEdge,
      child: InkWell(
        onTap: () => _showDetails(context),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [_buildPoster(), _buildInfo()],
        ), // Column
      ), // InkWell
    ); // Card
  }
}
```

Обратите внимание: `onTap` вызывает отдельный метод `_showDetails` — это избавит нас от дублирования кода. Методы `_buildPoster()` и `_buildInfo()` мы добавим на следующих шагах.

InkWell — виджет, который перехватывает нажатие и показывает анимацию «волны» в стиле Material Design.

Далее нам нужно с вами создать следующую структуру файлов:

```
class AnimeCard extends StatelessWidget {
  const AnimeCard({super.key, required this.anime});
  final Anime anime;

  @override Widget build(BuildContext context) {...}
  void _showDetails(BuildContext context) {...}
  Widget _buildPoster() {...}
  Widget _buildInfo() {...}
  Widget _buildTitleRow() {...}
  Widget _buildGenre() {...}
  Widget _buildRating() {...}
  Widget _buildDescription() {...}
}
```

Каждый метод — одна ответственность.

Шаг 1.2 — Виджет постера `_buildPoster()`

Добавьте в класс `AnimeCard` приватный метод для отображения изображения:

```
Widget _buildPoster() {  
  return Image.asset(  
    anime.imagePath,  
    height: 550,  
    width: double.infinity,  
    fit: BoxFit.cover,  
  );  
}
```

fit: BoxFit.cover — изображение масштабируется так, чтобы полностью заполнить отведённое пространство, без пустых полей по краям. Это аналог `ScaleType.centerCrop` в Android.

Шаг 1.3 — Блок с информацией `_buildInfo()`

Добавьте метод, который возвращает `Padding` с данными об аниме. Внутри — `Column` с тремя вспомогательными методами:

```
Widget _buildInfo() {  
  return Padding(  
    padding: const EdgeInsets.all(12),  
    child: Column(  
      crossAxisAlignment: CrossAxisAlignment.start,  
      children: [  
        _buildTitleRow(),  
        const SizedBox(height: 4),  
        _buildGenre(),  
        const SizedBox(height: 8),  
        _buildRating(),  
        const SizedBox(height: 8),  
        _buildDescription(),  
      ],  
    ), // Column  
  ); // Padding  
}
```


Шаг 1.3a — Название и год _buildTitleRow()

```
Widget _buildTitleRow() {
  return Row(
    mainAxisAlignment: MainAxisAlignment.spaceBetween,
    children: [
      Expanded(
        child: Text(
          anime.title,
          style: const TextStyle(
            fontSize: 18,
            fontWeight: FontWeight.bold,
          ), // TextStyle
        ), // Text
      ), // Expanded
      Text(
        '${anime.year}',
        style: const TextStyle(
          color: Colors.grey,
          fontSize: 14,
        ), // TextStyle
      ), // Text
    ],
  ); // Row
}
```

MainAxisAlignment.spaceBetween — название прижато к левому краю, год к правому. **Expanded** не даёт длинному названию «вытолкнуть» год за пределы строки.

Шаг 1.3b — Жанр _buildGenre()

```
Widget _buildGenre() {
  return Text(
    anime.genre,
    style: const TextStyle(
      color: Colors.deepPurple,
      fontSize: 13,
    ), // TextStyle
  ); // Text
}
```

Шаг 1.3c — Рейтинг `_buildRating()`

```
Widget _buildRating() {
  return Row(
    children: [
      const Icon(
        Icons.star,
        color: Colors.amber,
        size: 18,
      ), // Icon
      const SizedBox(width: 4),
      Text(
        anime.rating.toStringAsFixed(1),
        style: const TextStyle(
          fontWeight: FontWeight.bold,
          fontSize: 14,
        ), // TextStyle
      ), // Text
    ],
  ); // Row
}
```

`Icons.star` — встроенная иконка Material Design, не требует подключения сторонних пакетов.

Шаг 1.3d — Описание `_buildDescription()`

```
Widget _buildDescription() {
  return Text(
    anime.description,
    style: const TextStyle(
      fontSize: 13,
      color: Colors.black87,
    ), // TextStyle
    maxLines: 2,
    overflow: TextOverflow.ellipsis,
  ); // Text
}
```

`maxLines: 2 + TextOverflow.ellipsis` — текст обрезается до двух строк с многоточием. Это стандартный паттерн для карточек в списках.

Шаг 1.4 — Метод для `SnackBar _showDetails()`

Добавьте единый обработчик нажатия — теперь он один на всю карточку:

```
void _showDetails(BuildContext context) {  
  ScaffoldMessenger.of(context).showSnackBar(  
    SnackBar(  
      content: Text(anime.description),  
      duration: const Duration(seconds: 3),  
    ), // SnackBar  
  );  
}
```

`ScaffoldMessenger.of(context)` находит ближайший `Scaffold` в дереве виджетов и показывает всплывающее сообщение внизу экрана. `context` передаётся из `build()`. Именно поэтому он является параметром метода.

Теперь нажатие работает на всю карточку через `InkWell`, а отдельная кнопка `TextButton` больше не нужна.

Виджет / параметр	Назначение
Padding	Добавляет отступы вокруг дочернего виджета
CrossAxisAlignment.start	Выравнивание по левому краю в <code>Column</code>
MainAxisAlignment.spaceBetween	Элементы по краям строки — название слева, год справа
Expanded	Растягивает дочерний виджет на всё доступное пространство в <code>Row</code>
Icons.star	Встроенная иконка <code>Material Design</code> — без картинок
maxLines + TextOverflow.ellipsis	Обрезает длинный текст многоточием

 **СКРИНШОТ 3:** Скриншот готовой карточки `AnimeCard` в VS Code. Сохраните в `img/step3_ВашаФамилия.png`

Выполните в терминале:

git add .

git commit -m "Add AnimeCard widget"

Часть 4. Список `ListView.builder`

Шаг 1. Почему не `Column`?

На прошлых лабораторных мы использовали `Column` чтобы разместить виджеты вертикально. Почему бы не сделать то же самое для списка аниме?

Проблема в том, что `Column` создаёт **все дочерние виджеты сразу** — даже те, которые за пределами экрана. Для 10 элементов это не страшно, но для 100 или 1000 — приложение будет тормозить и расходовать лишнюю память.

`ListView.builder` решает эту проблему: он создаёт только те карточки, которые сейчас видны на экране. Когда пользователь скроллит — создаются новые, а те что ушли за край — удаляются. Это называется **виртуализация списка**. Аналог `RecyclerView` в Android.

	Column	ListView.builder
Создаёт виджеты	Все сразу	Только видимые
Скролл	Нет (нужен <code>SingleChildScrollView</code>)	Встроенный
Большие списки	❌ Тормозит	✅ Эффективно
Малые статичные списки	✅ Просто	Избыточно

Шаг 2. Создаём `AnimeList`

Создайте файл `lib/anime_list.dart`:

```
import 'package:anime_catalog/anime_card.dart';
import 'package:anime_catalog/models/anime_data.dart';
import 'package:flutter/material.dart';

class AnimeList extends StatelessWidget {
  const AnimeList({super.key});

  @override
  Widget build(BuildContext context) {
    return ListView.builder(
      padding: EdgeInsets.all(12),
      itemCount: animeList.length,
      itemBuilder: (context, index) {
        return Padding(
          padding: EdgeInsets.only(bottom: 16),
          child: AnimeCard(anime: animeList[index]),
        ); // Padding
      },
    ); // ListView.builder
  }
}
```

Разберём параметры `ListView.builder`:

- `itemCount` — сколько элементов в списке. Flutter использует это число, чтобы знать когда список заканчивается.

- **itemBuilder** — функция, которая строит виджет для каждого элемента. Принимает `context` и `index` (индекс текущего элемента) — вызывается только для видимых элементов.
- **padding** — отступы вокруг всего списка.

💡 `animeList[index]` — обращение к элементу списка по индексу. Точно так же как в Kotlin и C#. `index` начинается с 0 и идёт до `animeList.length - 1`.

Шаг 3. Подключаем в `main.dart`

Откройте `lib/main.dart`, удалите весь код и напишите:

```
import 'package:anime_catalog/anime_list.dart';
import 'package:flutter/material.dart';

Run | Debug | Profile
void main() {
  runApp(
    MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(
          seedColor: Colors.deepPurple,
        ), // ColorScheme.fromSeed
      ), // ThemeData
      home: Scaffold(
        appBar: AppBar(
          title: Text(
            'Аниме-каталог 🇯🇵',
            style: TextStyle(fontWeight: FontWeight.bold),
          ), // Text
          backgroundColor: Colors.deepPurple,
          foregroundColor: Colors.white,
        ), // AppBar
        body: AnimeList(),
      ), // Scaffold
    ), // MaterialApp
  );
}
```

Эмодзи для вставки - 🇯🇵

Здесь появляется новое — `AppBar`. Это верхняя панель приложения с заголовком. `Scaffold` принимает его через параметр `appBar`.

Также добавлена `theme` — глобальная тема приложения. `ColorScheme.fromSeed()` генерирует полную цветовую схему Material Design на основе одного цвета. Карточки, кнопки и другие виджеты автоматически подхватят цвета из темы.

Запустите приложение — вы увидите прокручиваемый список карточек аниме.



СКРИНШОТ 4: Скриншот браузера с работающим списком аниме-карточек.

Сохраните в `img/step4_ВашаФамилия.png`

Выполните в терминале:

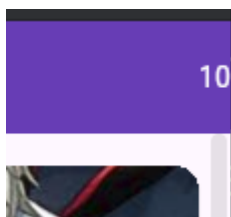
`git add .`

`git commit -m "Add AnimeList with ListView.builder and connect to main"`

Самостоятельное задание

Задание 1. Счётчик в AppBar

Добавьте в **AppBar** справа текст или иконку, которая показывает количество аниме в списке:



Для этого используйте параметр `actions` у **AppBar**:

```
appBar: AppBar(  
  title: Text(  
    'Аниме-каталог 🏳️‍🌈',  
    style: TextStyle(fontWeight: FontWeight.  
  ), // Text  
  backgroundColor: Colors.deepPurple,  
  foregroundColor: Colors.white,  
  actions: [REDACTED],  
), // AppBar
```

Подсказка №1: `animeList.length` возвращает количество элементов.

Подсказка №2: `actions` принимает список виджетов — попробуйте `Text` или `Padding` с `Text` внутри

Задание 2. Своя тематика

Замените аниме на любую другую тематику которая вам интересна: игры, фильмы, книги, музыкальные альбомы. Создайте свою модель данных с полями которые подходят для выбранной темы, подготовьте картинки и напишите данные.

Задание 3. README.md

Оформите **README.md** в репозитории **Flutter_Lab4**:

- Название лабораторной и номер
- Фамилия, имя, группа, дата сдачи
- Что изучили (3–5 пунктов своими словами)
- Скриншот финального приложения
- Инструкция по запуску



СКРИНШОТ 5: Скриншот финального приложения из Задания №2. Сохраните в **img/step5_ВашаФамилия.png**

Выполните в терминале:

git add .

git commit -m "added README description and completed labs"

git push